

Roteiro de Prática 3 — versão C++

Lista Compartilhada entre Threads, com MSYS2 e g++

Quando duas threads escrevem na mesma estrutura ao mesmo tempo

Unidade curricular: Programação Concorrente e Distribuída · Aula 02

Ambiente: MSYS2 (UCRT64) com g++ · Windows

Duração estimada: 60 minutos (com a instalação)

Objetivos de aprendizagem

Instalar e usar o compilador g++ pelo MSYS2; observar experimentalmente uma **condição de corrida**; identificar a **região crítica**; aplicar exclusão mútua com `std::mutex`; e reconhecer o **vazamento de memória** que a corrida provoca.

O que você vai fazer. Construir uma lista encadeada em C++ e entregá-la a várias threads ao mesmo tempo. O resultado vai ser desastroso — e é essa a intenção. Você vai **ver** milhares de dados desaparecerem, entender por quê, e então consertar com um *mutex*. E vai descobrir, na Parte 6, um segundo estrago que a corrida provoca e que passa despercebido.

Instalando o MSYS2 e o g++

Parte 0

O Windows não vem com compilador C++. O **MSYS2** é um ambiente que traz o **g++**, o mesmo compilador usado em Linux, em versão atual e com um gerenciador de pacotes para mantê-lo atualizado. Se você já instalou o MSYS2 na prática de pilha, pule para o passo 4 — mas **leia o aviso** logo abaixo.

- 1 Acesse **msys2.org** e baixe o instalador (um arquivo como `msys2-x86_64-data.exe`). Baixe sempre do site oficial. Execute e avance com as opções padrão.
- 2 No menu Iniciar, procure e abra **MSYS2 UCRT64**. Abre um terminal escuro. Atualize os pacotes:

```
$ pacman -Syu
```

Confirme com **Y** quando perguntar. **O terminal pode fechar sozinho ao final — isso é normal.** Reabra o **MSYS2 UCRT64** e rode o mesmo comando de novo, para concluir.

- 3 Instale o compilador (confirme com Enter quando perguntar):

```
$ pacman -S mingw-w64-ucrt-x86_64-gcc
```

- 4 Confirme que funcionou:

```
$ g++ --version
```

COMO VERIFICAR

Deve aparecer algo como `g++ (Rev, Built by MSYS2 project) 15.x.x`. Se aparecer `command not found`, veja o aviso abaixo — é quase sempre a mesma causa.

ATENÇÃO

O MSYS2 instala **vários** atalhos no menu Iniciar: UCRT64, MINGW64, MSYS, CLANG64. Eles são ambientes diferentes, **cada um com seu próprio PATH**. O erro clássico é instalar o compilador no UCRT64 e depois abrir o terminal **MSYS** comum, onde o `g++` não existe. **Use sempre o atalho "MSYS2 UCRT64"**.

ENTENDA

Materiais mais antigos (inclusive o nosso roteiro de pilha) mandam abrir "MSYS2 MinGW 64-bit" e instalar `mingw-w64-x86_64-gcc`. Isso ainda funciona, mas o projeto MSYS2 tornou o **UCRT64** o ambiente padrão e anunciou a **depreciação do MINGW64 em março de 2026**. Para instalações novas, use UCRT64. Se a sua máquina já tem o MINGW64 funcionando, pode continuar com ele nesta prática — só não misture bibliotecas dos dois.

Navegando até sua pasta

No MSYS2, o disco C: do Windows aparece como `/c/`. Crie e entre numa pasta para a prática:

```
$ mkdir -p /c/PCD/aula02
$ cd /c/PCD/aula02
```

A lista sem proteção**Parte 1**

Crie o arquivo `corrida.cpp` (use o Notepad++, o VS Code, ou o editor que preferir — salve na pasta acima). Comece pelo nó e pela lista:

```
#include <iostream>
#include <thread>
#include <vector>
#include <string>
using namespace std;

struct No {
    string dado;
    No* proximo;
    No(string d) : dado(d), proximo(nullptr) {}
};

class ListaInsegura {
private:
    No* inicio = nullptr;
    int tamanho = 0;
public:
    void inserir(string dado) {
        No* novo = new No(dado);
        No* antigoInicio = inicio;          // 1. LE o inicio atual
        this_thread::yield();                // 2. janela (explicada adiante)
        novo->proximo = antigoInicio;
        inicio = novo;                      // 3. ESCREVE o novo inicio
        tamanho++;                          // 4. conta
    }

    int getTamanho() { return tamanho; }

    // conta percorrendo no a no – a verdade sobre a lista
    int contarDeVerdade() {
        int c = 0; No* a = inicio;
        while (a != nullptr) { c++; a = a->proximo; }
        return c;
    }
};
```

ENTENDA

A lista guarda apenas o **ponteiro** para o primeiro nó (`No* inicio`). Todos os demais são alcançados seguindo a corrente: cada nó guarda o endereço do próximo, e o último aponta para `nullptr`. A seta `->` acessa o conteúdo apontado por um ponteiro: `a->proximo` significa "o campo proximo do nó que `a` aponta".

ATENÇÃO

Não esqueça o **ponto-e-vírgula depois da chave que fecha a classe** (`};`). Em C++ a declaração de uma classe termina com ponto-e-vírgula, e esquecer-lo é o tropeço mais comum — a mensagem de erro do g++ não ajuda nada a descobrir a causa.

Soltando as threads

Parte 2

Continue no mesmo arquivo, agora o `main`:

```
int main() {
    const int THREADS = 4, INSERCOES = 2000;
    ListaInsegura lista;
    vector<thread> ts;

    for (int i = 0; i < THREADS; i++) {
        ts.push_back(thread([&lista, i, INSERCOES]() {
            for (int j = 0; j < INSERCOES; j++)
                lista.inserir("T" + to_string(i) + "-" + to_string(j));
        }));
    }

    for (auto& t : ts) t.join();    // espera todas terminarem

    int esperado = THREADS * INSERCOES;
    cout << "Esperado      : " << esperado << endl;
    cout << "Contador tamanho : " << lista.getTamanho() << endl;
    cout << "Contando os nos   : " << lista.contarDeVerdade() << endl;
    cout << "Nos PERDIDOS      : " << (esperado - lista.contarDeVerdade()) << endl;
    return 0;
}
```

ENTENDA

O trecho `[&lista, i, INSERCOES]() { ... }` é uma **lambda**: uma função sem nome, criada ali mesmo. Os colchetes dizem o que ela captura de fora: a lista **por referência** (`&lista` — todas as threads mexem na mesma), e `i` e `INSERCOES` por cópia (cada thread tem o seu). Se você capturasse a lista por cópia, cada thread teria a sua própria e não haveria concorrência nenhuma.

Compilando

```
$ g++ -std=c++11 -pthread corrida.cpp -o corrida
$ ./corrida
```

ATENÇÃO

O `-pthread` é **obrigatório** para usar threads. Sem ele, o programa pode nem compilar, ou compilar e falhar de forma estranha ao rodar. E o `-std=c++11` garante que `std::thread` esteja disponível.

COMO VERIFICAR

Uma execução real:

```
Esperado      : 8000
Contador tamanho : 8000
Contando os nos   : 2002
Nos PERDIDOS      : 5998
```

O contador afirma 8000, mas percorrendo a lista só existem 2002 nós. Rode três ou quatro vezes: os números variam. Essa variação é a assinatura de uma condição de corrida.

Por que quase 6 mil nós sumiram

Parte 3

Acompanhe duas threads executando `inserir` ao mesmo tempo. A lista começa com o nó **A** no início:

Momento	Thread 1 (insere B)	Thread 2 (insere C)
1	lê <code>inicio</code> → guarda A	

2	— trocada pelo SO —	lê inicio → guarda A também
3		C→proximo = A; inicio = C
4	B→proximo = A; inicio = B	
5	Resultado: inicio aponta para B, que aponta para A. O nó C sumiu da lista — nenhum ponteiro leva até ele.	

As duas threads leram o mesmo **inicio** e cada uma escreveu por cima da outra. Repita isso milhares de vezes e você tem os 6 mil nós perdidos.

ATENÇÃO

O trecho entre a **leitura** e a **escrita** de um dado compartilhado chama-se **região crítica**. Onde há região crítica acessada por várias threads, é preciso garantir que **só uma entre por vez**.

ENTENDA

Sobre o `this_thread::yield()`: ele **não cria** o problema, apenas alarga a janela entre a leitura e a escrita, tornando a falha consistente. Sem ele, o erro aparece raramente — o que é ainda mais perigoso, porque o programa passa nos testes e quebra em produção. A prova virá na Parte 4: a versão corrigida mantém o `yield` e não perde nada.

A correção com mutex

Parte 4

Um **mutex** (de *mutual exclusion*) é um cadeado: enquanto uma thread o segura, as outras esperam. Crie `seguro.cpp` — igual ao anterior, com as três linhas destacadas:

```
#include <mutex>                                // 1. o cabeçalho

class ListaSegura {
private:
    No* inicio = nullptr;
    int tamanho = 0;
    mutex trava;                                // 2. o cadeado
public:
    void inserir(string dado) {
        No* novo = new No(dado);                // fora: cada thread tem o seu
        lock_guard<mutex> lock(trava);           // 3. tranca aqui
        No* antigoInicio = inicio;
        this_thread::yield();                    // a MESMA janela de antes
        novo->proximo = antigoInicio;
        inicio = novo;
        tamanho++;
    }                                            // destranca sozinho ao sair

    int getTamanho() { lock_guard<mutex> lock(trava); return tamanho; }

    int contarDeVerdade() {
        lock_guard<mutex> lock(trava);
        int c = 0; No* a = inicio;
        while (a != nullptr) { c++; a = a->proximo; }
        return c;
    }
};
```

```
$ g++ -std=c++11 -pthread seguro.cpp -o seguro
$ ./seguro
```

COMO VERIFICAR

Agora, em todas as execuções:

```
Esperado      : 8000
Contador tamanho : 8000
Contando os nos : 8000
Nos PERDIDOS   : 0
```

O `yield` continua no código. A janela não sumiu — o que mudou é que nenhuma outra thread consegue entrar enquanto uma está dentro. Isso prova que o problema nunca foi o `yield`: era a falta de exclusão mútua.

ENTENDA

O `lock_guard` tranca ao ser criado e **destranca sozinho** quando sai de escopo — mesmo se o método terminar por uma exceção. É o padrão RAII do C++, e é por isso que raramente se chama `trava.lock()` e `trava.unlock()` à mão: se você esquecer o `unlock`, o programa trava para sempre.

Ligação com a aula

Parte 5

Tudo que você acabou de escrever tem nome nos conceitos vistos em sala. Esta tabela mostra onde cada conceito da aula apareceu no seu código:

Conceito da aula	Onde ele aparece nesta prática
Thread	Cada <code>std::thread</code> criada no laço do <code>main</code> . As quatro executam o mesmo trecho ao mesmo tempo, de forma independente.
Esperar terminar	O <code>t.join()</code> : o programa principal fica parado até a thread acabar. Sem ele, os totais seriam impressos antes das threads terminarem.
Recurso compartilhado	O objeto <code>lista</code> , capturado por referência (<code>&lista</code>) na lambda. É o mesmo objeto para todas as threads — daí o conflito.
Região crítica	O trecho entre ler o <code>inicio</code> e escrever o novo valor. É onde duas threads não podem estar ao mesmo tempo.
Condição de corrida	O resultado da Parte 2: nós perdidos e contador errado, variando a cada execução.
Exclusão mútua	O <code>std::mutex</code> com <code>lock_guard</code> da Parte 4: garante uma thread por vez na região crítica.
Monitor / cadeado	O atributo <code>mutex trava</code> . Enquanto uma thread o segura, as demais ficam bloqueadas esperando a vez.

LIGAÇÃO COM A AULA

Guarde a ideia central, que vale para qualquer linguagem e para o resto da disciplina: **onde existe um recurso compartilhado entre threads, existe uma região crítica; e toda região crítica precisa de exclusão mútua**. As ferramentas mudam de nome conforme a linguagem, mas o problema e a solução são sempre esses. Identificar *onde* está a região crítica é a parte difícil — e é o ofício de quem programa concorrência.

O segundo estrago: memória vazada

Parte 6

Volte à Parte 3 e olhe de novo o nó C, que sumiu da lista. Pergunte-se: **o que aconteceu com a memória dele?**

O nó foi criado com `new`, o que reserva um espaço de memória. Depois, por causa da corrida, o ponteiro que levava até ele foi sobrescrito. O espaço **continua reservado**, mas agora **nenhum ponteiro leva até ele** — então é impossível chamar `delete` para liberá-lo. Essa memória fica presa até o programa terminar. Isso se chama **vazamento de memória** (*memory leak*).

ATENÇÃO

A condição de corrida causa **dois** estragos de uma vez: perde os dados e vaza memória. Naquela execução foram 5998 nós vazados. Num servidor rodando por semanas, esse vazamento cresce até o programa consumir toda a memória da máquina e ser derrubado pelo sistema — dias depois, sem relação aparente com a causa.

ENTENDA

Vale a regra de ouro do gerenciamento de memória em C++: **quem aloca com `new` é responsável por liberar com `delete`**. O que torna este caso traiçoeiro é que ninguém escreveu código errado de propósito — a responsabilidade se perdeu sozinha, no meio da disputa entre as threads.

Exercícios

Parte 7

1. Rode a versão insegura **cinco vezes** e anote os resultados numa tabela. Faça o mesmo com a segura. Escreva um parágrafo comparando: o que muda entre execuções em cada caso, e por quê?

2. Remova o `this_thread::yield()` da versão insegura e rode dez vezes. Os erros ainda aparecem? Com que frequência? Explique por que um erro que aparece às vezes é mais perigoso que um que aparece *sempre*.

3. Meça o tempo das duas versões com `chrono::steady_clock`. Qual é mais lenta, e por quanto? Esse é o custo da sincronização — e vale a pena?

4. Escreva um destrutor `~ListaSegura()` que percorra a lista liberando cada nó com `delete`. Sem ele, mesmo a versão correta vaza toda a memória ao final do programa.

5. Troque o método inteiro por um bloco menor: mantenha o `new No(dado)` fora da trava (como já está) e verifique que continua correto. Depois tente trancar *apenas* o `tamanho++` e veja o que acontece. Por que não basta?

6. Desafio: compile com `g++ -std=c++11 -pthread -static corrida.cpp -o corrida_static`. O executável fica bem maior. Copie-o para um pen drive e rode em outra máquina Windows, sem MSYS2 instalado. Por que a versão estática funciona lá e a normal não?